Open in app ↗

Medium Search Write

Member-only story

Analyze NBA Stats with the NBA API and Python

A Deep Dive into Data Analysis and Visualizations in Python

Ben Ballard 6 min read · Dec 11, 2023

92 1

This post is a walk-through on how I created a process using python to pull NBA data through the NBA API and analyze player career stats like field goal percentages, points, rebounds and assists. In particular, I'll focus on how I pulled from the NBA API, the pandas.

Basketball and Data Science!

Gathering the Data

The cornerstone of this process is pulling the player career data. To easily navigate this, I recommend using my NBA API Buddy. I designed this CustomGPT with an in-depth understanding of the NBA API, encompassing essential elements like understanding the API endpoints as well as knowing Player and Team IDs. If your query deviates from the examples provided, don't hesitate to test it out! The NBA API Buddy is equipped to handle a all API inquiries, making your data gathering both efficient and user-friendly.

The first step is identifying what player you'd like to analyze. Then use the NBA API Buddy to identify the Player ID. Replace the current ID '202681' with whatever the new playerID is.

The code utilizes the `nba_api.stats.endpoints` module, specifically tapping into `playercareerstats`. This function will then fetch the comprehensive career statistics of your chosen player.

```
from nba_api.stats.endpoints import playercareerstats

# Fetching career statistics for Player of Choice using his player ID
player_career = playercareerstats.PlayerCareerStats(player_id='202681')
player_career_df = player_career.get_data_frames()[0]

# Extracting the seasons of player of choice
seasons_played = player_career_df['SEASON_ID'].unique()
print(seasons_played.tolist())
```

The above code will print out the number of seasons the NBA player you chose played in a list like this: *['2019–20', '2020–21', '2021–22', '2022–23', '2023–24']*. Copy the years output and replace the list of seasons below.

```
import matplotlib.pyplot as plt
import pandas as pd
from nba_api.stats.endpoints import playergamelog

# Initialize an empty DataFrame to store all game logs
all_seasons_logs_df = pd.DataFrame()

# List of seasons to loop through (update this list as needed)
seasons = ['2019-20', '2020-21', '2021-22', '2022-23', '2023-24']

# Fetch game logs for each season and add a 'SEASON' column
for season in seasons:
    player_logs = playergamelog.PlayerGameLog(player_id='202681', season=season)
    season_logs_df = player_logs.get_data_frames()[0]
    season_logs_df['SEASON'] = season
    all_seasons_logs_df = pd.concat([all_seasons_logs_df, season_logs_df], ignor
```

After running the above, *all_season_logs_df* dataframe will include all game data for the player of interest. The table will includes many more stats, but the table below is a small example of what is contained in the data.

GAME_DATE	MATCHUP	WL	MIN	FGM	FGA	FG_PCT	FG3M	FG3A	FG3_PCT
2012-04-25	CLE vs. WAS	L	10	2	4	0.500	1	2	0.500
2012-04-23	CLE @ MEM	L	29	10	16	0.625	2	6	0.333
2012-04-22	CLE @ SAS	L	30	8	18	0.444	1	3	0.333

Sample of the Season Logs for Player from NBA API

Data Processing

After pulling the data, the next steps involve processing the data for analysis and visualization. While the data comes pretty clean after the API pull, there was some processing I did to. First, ensure the GAME_DATE column is in a datetime format, and to do that you can use the pandas to_datetime method.

```
# Convert Game_Date to a datetime
all_seasons_logs_df['GAME_DATE'] = pd.to_datetime(all_seasons_logs_df['GAME_DATE
# Create Month_Year to faciliate Month/Date Analysis
all_seasons_logs_df['MONTH_YEAR'] = all_seasons_logs_df['GAME_DATE'].dt.to_perio
```

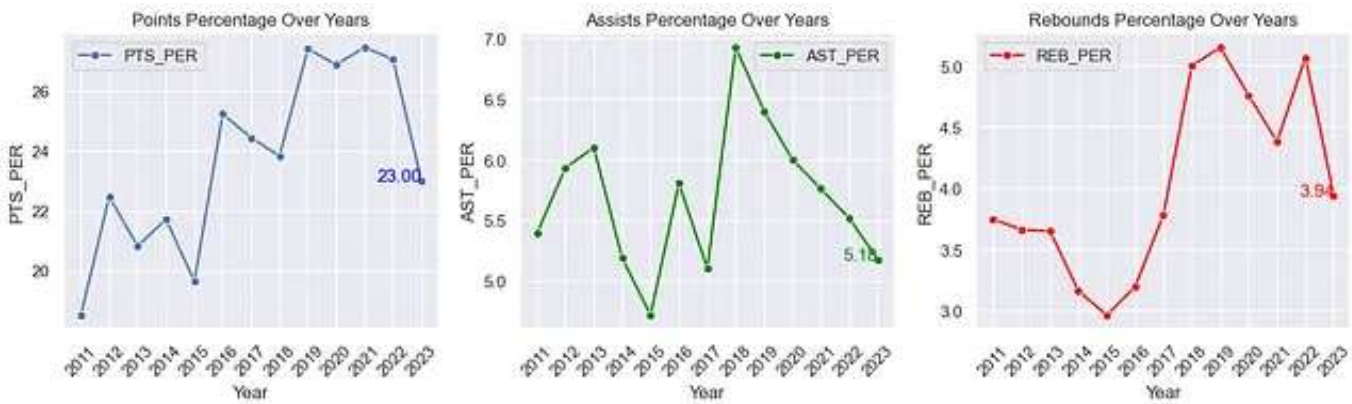
Creating summary tables is important for understanding the data and building narratives for story-telling. I used the following code to create a yearly aggregation of the game level stats to tell the story of a players' career.

```
#Aggregate game level data to yearly
yearly_stats = all_seasons_logs_df.groupby('YEAR').agg({
    'FGM': 'sum',
    'FGA': 'sum',
    'FG3M': 'sum',
    'FG3A': 'sum',
    'FTM': 'sum',
    'FTA': 'sum',
}).reset_index()
```

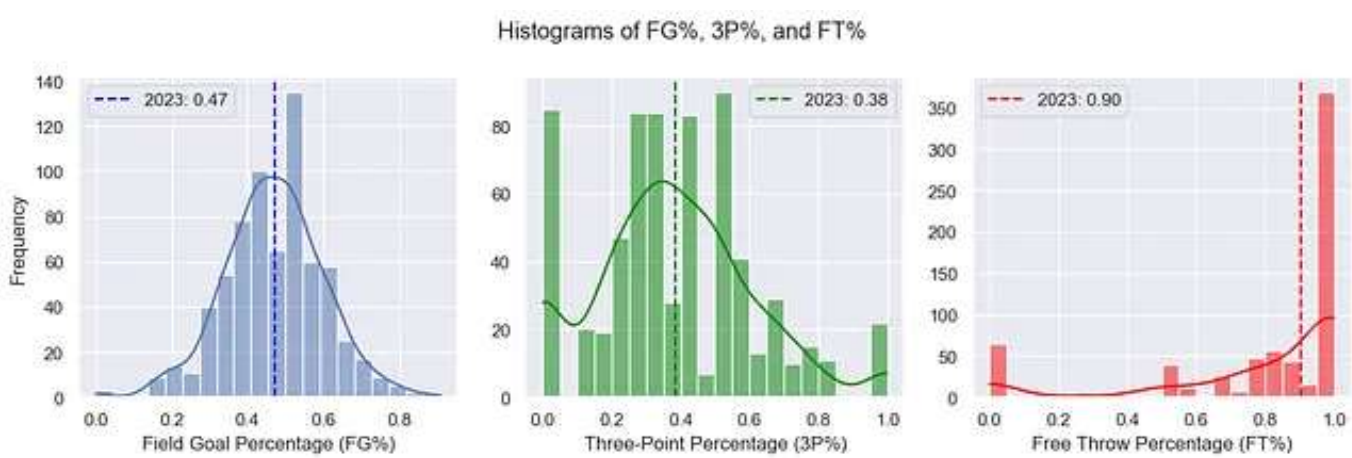
```
# This takes the field goals made and attempted to calculate field goal percent
yearly_stats['FG_PCT'] = yearly_stats['FGM'] / yearly_stats['FGA']
yearly_stats['FG3_PCT'] = yearly_stats['FG3M'] / yearly_stats['FG3A']
yearly_stats['FT_PCT'] = yearly_stats['FTM'] / yearly_stats['FTA']
yearly_stats['GAMES'] = lebron_data.groupby('YEAR')['Game_ID'].count().values
```

Data Visualization

The final part of this script is visualizing the data. There are many options to visualize the data from line charts, histograms, bar graphics. I used seaborn and matplotlib to create visualizations.

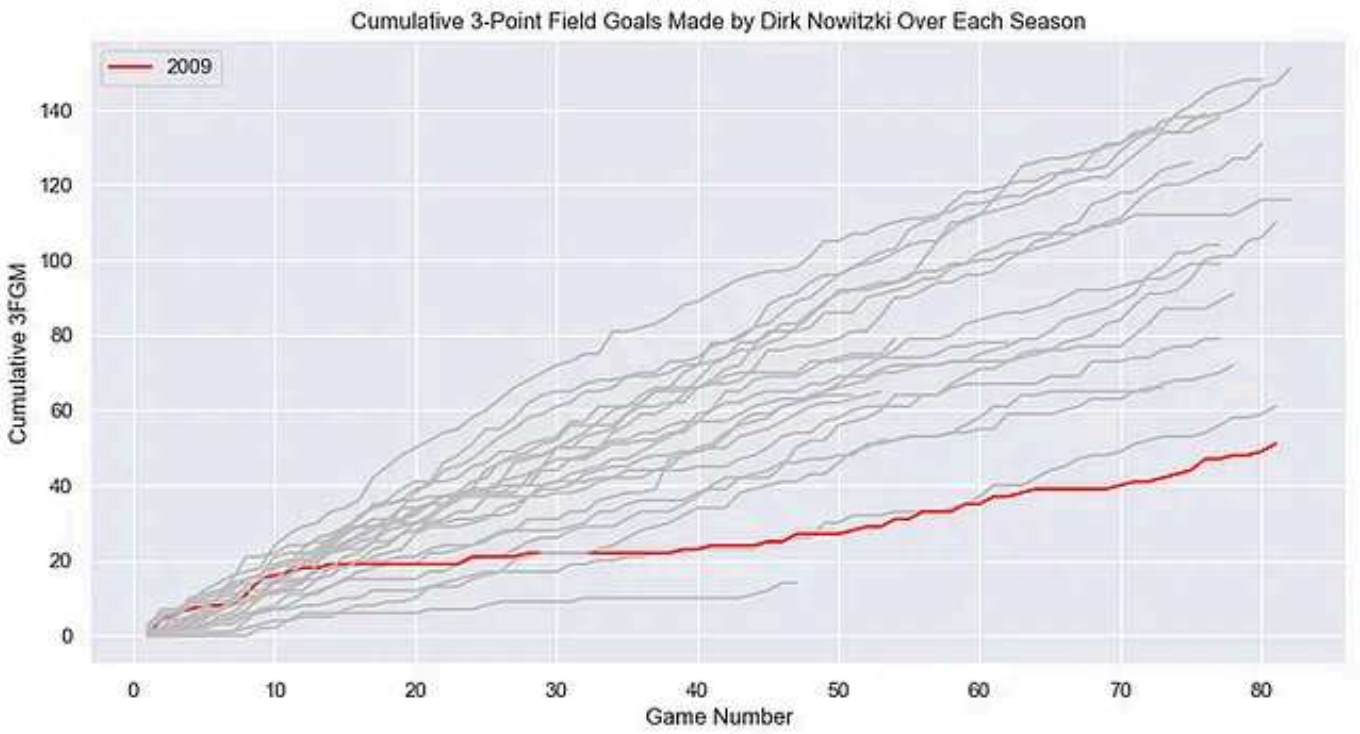


Example 1 of Visualizations. Line Grahps showing Points, Assists, and Rebounds



Example 2 of Visualizations. Histograms showing Field Goal Percentages by Category.

Finally, I'll present a visualization that I enjoyed creating. This graph tracks a specific stat — in this case, the number of three-pointers made per season — and compares it across different seasons. For a specific example, we'll look at Dirk Nowitzki's career. The visualization plots the cumulative count of three-pointers Dirk made each season, with the game number on the X-axis. To highlight a specific season, like 2009, I've used a red line. Interestingly, while 2009 was a career-high in three-point percentage for Dirk, it featured one of his lowest totals for made threes. This type of visualization offers a nuanced view of a player's performance over time.



In order to get that graphic to work, we must first create the cumulative sum of threes (FG3M) for each season. The code below does that for us.

```
# Calculate Cummulative Sum
all_seasons_logs_df['FG3M_CUMSUM'] = all_seasons_logs_df.groupby('YEAR')['FG3M']
```

```
all_seasons_logs_df['FGM_CUMSUM'] = all_seasons_logs_df.groupby('YEAR')['FGM'].c
```

Finally, this code snippet is designed to plot a distinct line for each season, representing the cumulative field goals made by the player. What makes this visualization particularly engaging is the use of color coding. I’ve chosen red for the 2023 season to make that pop for analysis, making it stand out against the others which will be grey. Additionally, to aid in readability and reference, only the year 2023 is labeled on the graph.

```
# Plotting a line for each season
for year in all_seasons_logs_df['YEAR'].unique():
    season_data = all_seasons_logs_df[all_seasons_logs_df['YEAR'] == year]

    # Plot a Particular Year of Interest Red
    # Using the team color for each season, and bright red for the year 2009
    color = 'red' if year == 2023 else 'silver'
    # Label only for the year 2023
    label = f'{year}' if year == 2023 else None
    plt.plot(season_data['Game_Number'], season_data['FGM_CUMSUM'], label=label,

plt.title('Cumulative Field Goals Made by Player Over Each Season')
plt.xlabel('Game Number')
plt.ylabel('Cumulative FG Made').
```

Alright, one last section to wrap up this post.

I plan to revisit this in a few days and work on a few areas. Specifically:

- **Dynamic Season List**: Instead of having the first block of code to output the season a player played and then hardcoding the seasons list, I would like to dynamically generate it based on the player's career data. This would make the script more flexible and reduce the need for manual updates.
- **Modularization**: I do some parts like manually updating graph titles and other running cells in a specific order in my notebook. I'd like to break down the notebook into functions for specific tasks (i.e. fetching data, processing data, and plotting). This will make the code more organized and reusable.
- Of course, **Documentation**: Adding more comments and explanations throughout the script will improve my readability and make it easier for me and others use your code. Once I clean it up, I plan to post to my github.

If you're interested, here's an example of a story where I utilized the above workflow to write about Luka Doncic's NBA MVP case.

Luka's MVP Level Season and Higher Scoring and Efficiency

As the NBA season unfolds, Luka Dončić's performance has been nothing short of spectacular, and his statistics position...

medium.com

Hope you found value in this. Please let me know. I'm always interested in other points of view and learning from others on how to improve my analysis process.

Data Science | NBA | Data Visualization | Data Analysis | Python

Written by Ben Ballard

203 followers · 365 following

Edit profile

Here for Data Science and Machine Learning. MS Data Science @UVA | Boomer | Mavs | Working on NBAanalytics.com.

Responses (1)

Ben Ballard

What are your thoughts?

Laura Bosco
Dec 11, 2023

This is so helpful - thank you!

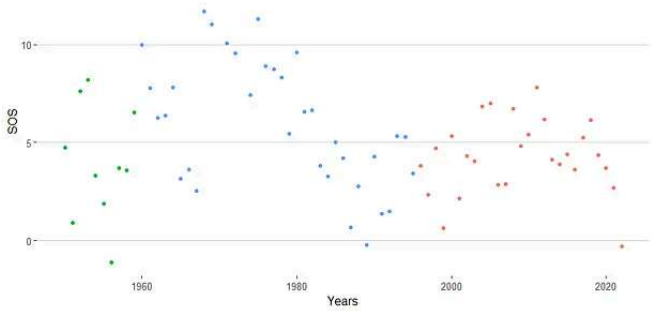
1 Reply

More from Ben Ballard

Ben Ballard

Lebron James Career-Year from Beyond the Arc

Dec 8, 2023 4



In Data Science Cowboy by Ben Ballard

OU Strength of Schedule Over Time

For my next analysis, I took a look at the strength of schedule variable. I used the R...

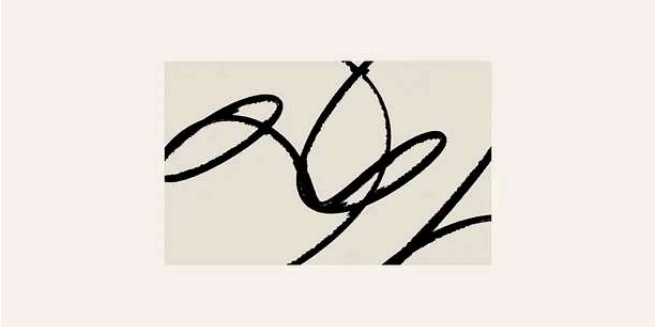
Sep 17, 2022

Ben Ballard

Unlocking Sports Betting with Python: The Odds API

Harnessing Live Data for Smarter Wagering

Dec 16, 2023 96



In Data Science Cowboy by Ben Ballard

The Strength of ggplot

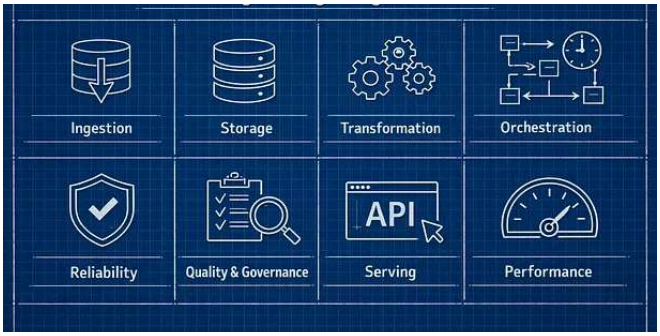
For those new to programming in R, ggplot is a powerful and widely used data visualizatio...

Jan 5, 2023 46 1

See all from Ben Ballard

Recommended from Medium

Will Lockett



In AWS in Plain English by Khushbu Shah

The AI Bubble Is About To Burst, But The Next Bubble Is Already...

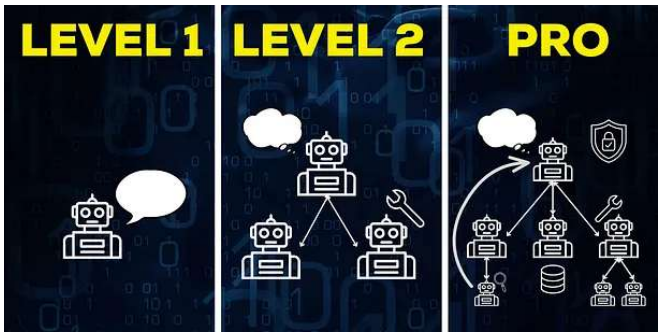
Techbros are preparing their latest bandwagon.

Sep 14, 2025 22K 946

Data Engineering Design Patterns You Must Learn in 2026

These are the 8 data engineering design patterns every modern data stack is built on....

Jan 5 898 19



In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

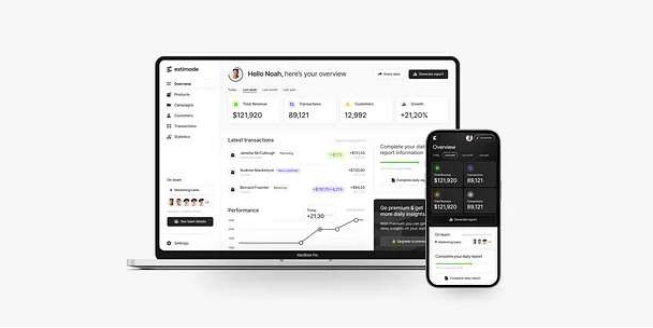
Dec 6, 2025 3.4K 116

In Artificial Corner by The PyCoach

The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these

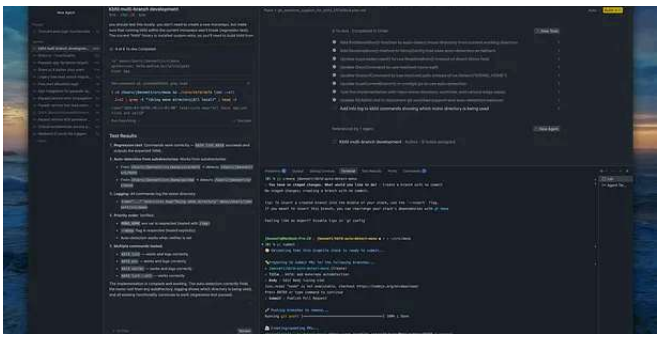
Dec 1, 2025 4.5K 246



Michal Malewicz

The End of Dashboards and Design Systems

Design is becoming quietly human again.



Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Nov 26, 2025 5.5K 212 Jan 18 2.6K 68

See more recommendations